AI-Driven Smart Inventory and Stock Flow Optimisation

1. Component Overview

The AI-Driven Smart Inventory and Stock Flow Optimisation component is part of the Smart Omni-Retail System. It is designed to support intelligent inventory planning and stock movement across multiple fashion retail stores.

The component analyses historical sales, product information, store-level inventory, promotions, weather conditions, calendar events, and previous demand patterns to predict future product demand. Based on these predictions, it identifies stock shortages, reorder requirements, excess inventory, and opportunities to transfer stock between stores.

The system also provides explainable recommendations to retail managers. Instead of only presenting a forecast value, it explains the inventory condition, identifies the most suitable source store, estimates the logistics cost, evaluates the transfer risk, and supports the complete approval and execution workflow.

The main goal of the component is to reduce stockouts, minimise overstock, improve product availability, reduce unnecessary purchasing, and support faster data-driven inventory decisions.

---

2. Problem Addressed

Multi-branch fashion retailers frequently experience unequal stock distribution. A product may be out of stock or nearly unavailable in one branch while the same product is overstocked in another branch.

Traditional inventory systems mainly record stock and sales transactions. They often do not provide:

- Future demand forecasting
- Early stock-shortage detection
- Overstock identification
- Store-to-store stock transfer recommendations
- Logistics cost analysis
- Recommendation confidence
- Risk assessment
- Explainable inventory decisions
- An integrated approval and execution workflow

As a result, inventory teams may depend on manual reports, spreadsheets, experience, and repeated communication between branches. This can delay decisions and increase the risk of lost sales, excess stock, unnecessary purchasing, and inefficient stock movements.

---

3. Main Objective

The main objective is to develop an AI-driven inventory intelligence system that predicts future demand, evaluates stock conditions, recommends appropriate inventory actions, and optimises stock movement between retail stores.

Specific Objectives

1. Predict future product demand for different stores and time periods.
2. Identify products that may require reordering.
3. Detect products with excess or slow-moving inventory.
4. Estimate inventory health and days on hand.
5. Identify the best source store for stock transfers.
6. Rank alternative source stores.
7. Estimate transfer distance and logistics cost.
8. calculate recommendation confidence and risk level.
9. Explain AI-generated inventory decisions in understandable language.
10. Provide a complete approval, rejection, cancellation, and execution workflow.
11. Automatically update inventory and transaction history after a stock transfer.

---

4. Main System Functions

4.1 Demand Forecasting

The forecasting module predicts future product demand using historical and contextual information.

The implemented forecasting capabilities include:

- Daily forecasting
- Monthly forecasting
- Quarterly forecasting
- Product-wise forecasting
- Store-wise forecasting
- Category-wise forecasting
- Brand-wise forecasting
- Gender-wise forecasting
- Forecast confidence score
- AI-generated explanation
- Inventory recommendation output

The forecasting model considers information such as:

- Historical sales
- Previous-day sales
- Previous-week sales
- Rolling sales average
- Product category
- Product subcategory
- Brand
- Gender
- Product price
- Promotion percentage
- Store
- Temperature
- Humidity
- Rainfall
- Holidays
- Festivals
- School periods
- Weekends
- Month
- Day
- Day of the week

The predicted demand is used as an input for inventory analysis and stock movement recommendations.

4.2 Inventory Intelligence

The inventory intelligence module compares predicted demand with current stock information.

It generates:

- Inventory health status
- Stock-level analysis
- Reorder prediction
- Overstock detection

- Days-on-hand estimation
- Shortage identification
- Inventory recommendation
- Priority level

The system can classify inventory conditions into business-oriented statuses such as:

- Healthy
- Low stock
- Reorder required
- Potential stockout
- Overstock
- Critical

These classifications help retail managers understand the inventory situation without manually analysing large amounts of data.

4.3 Smart Stock Recommendation Engine

When the system identifies a stock shortage, it checks whether the required quantity can be obtained from another store before recommending a new purchase.

The recommendation engine provides:

- Best source-store selection
- Alternative source-store ranking
- Available transfer quantity
- Transfer distance
- Estimated logistics cost
- Recommendation confidence
- Risk level
- Trade-off analysis
- Recommendation reason

The source-store selection considers factors such as:

- Available stock at the source store
- Predicted demand at the source store
- Safety stock requirement
- Distance between stores

- Transfer quantity
- Logistics cost
- Risk of creating a shortage at the source store

The system avoids transferring stock from a branch if the transfer could create another inventory problem at that branch.

4.4 Stock Movement Workflow

The stock movement module supports the full lifecycle of an inventory transfer recommendation.

The implemented workflow includes:

1. Generate recommendation
2. Review recommendation
3. Approve recommendation
4. Reject recommendation
5. Cancel recommendation
6. Execute transfer
7. Update source inventory
8. Update destination inventory
9. Create inventory transaction
10. Update movement history

When a transfer is executed, the system automatically:

- Deducts the transferred quantity from the source store
- Adds the transferred quantity to the destination store
- Creates an inventory transaction
- Records the transfer status
- Updates the movement history
- Stores execution-related information

This reduces manual stock adjustments and improves traceability.

4.5 Explainable AI

The system uses generative AI to convert technical forecasting and inventory outputs into understandable explanations.

The explanation may describe:

- Why demand is expected to increase or decrease
- Why a product requires reordering
- Why a product is considered overstocked
- Why a particular source store was selected
- What risks are associated with a transfer
- What action the manager should take

This improves transparency and makes the system more practical for non-technical retail users.

---

5. System Workflow

The overall workflow is as follows:

1. Product, sales, inventory, promotion, weather, event, and store data are collected.
2. The data are cleaned and transformed into model-ready features.
3. The XGBoost model predicts future demand.
4. The predicted demand is compared with current stock.
5. The inventory intelligence module calculates inventory health, days on hand, reorder requirements, and overstock conditions.
6. If a shortage is detected, the system searches for suitable source stores.
7. Candidate stores are evaluated based on stock availability, predicted demand, distance, logistics cost, and operational risk.
8. The best source store and alternative stores are ranked.
9. The system generates a recommendation with a confidence score, risk level, and explanation.
10. A retail manager reviews the recommendation.
11. The recommendation can be approved, rejected, or cancelled.
12. An approved recommendation can be executed.
13. Inventory records, transactions, and movement history are updated automatically.

---

6. System Architecture

The component follows a layered architecture.

Presentation Layer

The Flutter Web frontend provides:

- Inventory dashboard

- Forecasting screens
- Product and store selection
- Inventory-health views
- Stock recommendations
- Recommendation details
- Approval and rejection actions
- Transfer execution
- Movement history
- AI explanations

Application and API Layer

The FastAPI backend handles:

- Request validation
- Forecasting requests
- Inventory analysis
- Recommendation generation
- Source-store ranking
- Distance calculation
- Logistics cost calculation
- Recommendation workflow
- Inventory updates
- Transaction recording
- AI explanation generation
- Firebase communication

Machine-Learning Layer

The machine-learning layer contains:

- Daily demand forecasting model
- Monthly inventory intelligence model
- Model column definitions
- Feature metadata
- Prediction processing
- Confidence calculation

XGBoost was selected because it performs effectively on structured tabular data and can model complex relationships between historical demand, product attributes, environmental conditions, and calendar-based factors.

Data Layer

Firebase Firestore stores:

- Products
- Stores
- Inventory
- Sales
- Forecasts
- Recommendations
- Inventory transactions
- Stock movements
- Users
- Product images and URLs
- Model-related outputs

Firebase Authentication manages user access.

Explainable-AI Layer

Gemini is used to produce natural-language explanations and business recommendations based on the forecasting and inventory analysis outputs.

---

7. Technologies Used

Area	Technology
Frontend	Flutter Web
Frontend language	Dart
Backend	FastAPI
Backend language	Python
Database	Firebase Firestore
Authentication	Firebase Authentication
Machine learning	XGBoost Regressor

Area	Technology
Data processing	Pandas, NumPy
Model persistence	Pickle and JSON metadata files
Explainable AI	Gemini
Planned conversational assistant	Groq
API communication	REST APIs
Product-image hosting	GitHub Pages
Development tools	Visual Studio Code, Git and GitHub
Testing	FastAPI Swagger UI, API testing and functional workflow testing

---

8. Dataset and Data Scope

The developed dataset represents a multi-store fashion retail environment.

Stores

The implementation includes eight stores:

- CP001 – Nugegoda
- CP002 – Maharagama
- CP003 – Kandy
- CP004 – Kirulapane
- CP005 – Wattala
- CP006 – Colombo
- CP007 – Pelawatte
- CP008 – Malabe

Product Scope

The dataset contains approximately 1,000 products with attributes such as:

- Product ID
- Product name
- Category
- Subcategory
- Brand
- Colour

- Gender
- Price
- Unit cost
- Product-image URL

Brands

The included brands are:

- Emerald
- Hustle
- Miss Modano
- Modano
- King Street
- Libera

Main Data Features

The sales and forecasting data include:

- Date
- Store ID
- Product ID
- Category
- Units sold
- Price
- Discount
- Temperature
- Humidity
- Rainfall
- Weather condition
- Event
- Event type
- Holiday status
- Promotion status
- Festival status
- School-period status

- Weekend status
- Lag 1
- Lag 7
- Rolling mean of seven days
- Month
- Day
- Day-of-week number

---

9. Backend Implementation

The backend was developed using FastAPI and organised into separate services.

The main backend responsibilities include:

- Loading trained machine-learning models
- Validating incoming prediction requests
- Preparing one-hot-encoded model inputs
- Generating predictions
- Calculating confidence
- Retrieving product and store data
- Analysing inventory
- Generating recommendations
- Ranking source stores
- Managing recommendation status
- Executing stock transfers
- Updating Firebase records
- Generating AI explanations

Important endpoints implemented during development include:

- Health-check endpoint
- Forecasting and decision endpoint
- Dropdown-data endpoint
- Product and store information endpoint
- Chat or explanation endpoint
- Recommendation-generation endpoints

- Recommendation workflow endpoints

The exact final endpoint names should be copied from the current FastAPI Swagger documentation before publication.

---

10. Frontend Implementation

The frontend was developed using Flutter Web.

The user interface provides:

- Dashboard summary
- Product selection
- Store selection
- Forecast period selection
- Forecast result presentation
- Confidence display
- Inventory-health information
- Reorder and overstock status
- Source-store recommendations
- Alternative store comparison
- Cost and distance information
- Risk and confidence indicators
- AI-generated explanations
- Approval and rejection controls
- Transfer execution
- Movement-history tracking

The interface was designed to present complex analytical outputs in a form suitable for retail managers and inventory users.

---

11. Database Implementation

Firebase Firestore is used as the shared group database.

The inventory component uses or requires collections for:

- Products
- Stores
- Inventory

- Sales
- Forecasts
- Recommendations
- Inventory transactions
- Movement history
- Users

The final research paper should use the exact collection names currently implemented in the group Firebase project.

Common identifiers are used to support integration:

- product_id
- store_id
- customer_id, where relevant
- recommendation_id
- transaction_id

Product identifiers and image URLs also support integration with the Smart Mirror and personalised marketing components.

---

12. Integration with the Smart Omni-Retail System

The inventory component supports the other research modules by providing:

- Product availability
- Store-level stock information
- Demand forecasts
- Reorder status
- Overstock information
- Suggested stock movements
- Product and store information

Possible integration examples include:

Smart Mirror Integration

The Smart Mirror can obtain:

- Product details
- Product images
- Available stock

- Store availability

This can prevent the system from recommending unavailable fashion products.

Personalised Marketing Integration

The marketing module can use:

- Stock availability
- Overstock information
- Predicted demand
- Product information
- Promotion-related demand patterns

This can help ensure that marketing campaigns promote relevant and available products.

Customer Intelligence Integration

Customer interactions, preferences, responses, and marketing engagement may provide additional demand signals for future forecasting improvements.

---

13. Implementation Results

The following system functions have been successfully implemented:

- Daily demand forecasting
- Monthly demand forecasting
- Quarterly demand forecasting
- Forecast confidence calculation
- AI-generated explanations
- Inventory-health analysis
- Reorder prediction
- Overstock detection
- Days-on-hand calculation
- Best source-store selection
- Alternative source-store ranking
- Distance calculation
- Logistics cost calculation
- Recommendation confidence
- Risk-level calculation

- Trade-off analysis
- Recommendation generation
- Recommendation approval
- Recommendation rejection
- Recommendation cancellation
- Stock-transfer execution
- Automatic inventory updates
- Inventory-transaction creation
- Movement-history updates
- Firebase integration
- Flutter frontend integration
- FastAPI backend integration

---

14. Machine-Learning Results

During development, the daily forecasting model produced approximately the following results:

Metric	Recorded Value
Mean Absolute Error	0.5418
Root Mean Squared Error	0.9436
Coefficient of Determination	0.9497

These values indicate that the model explained a high proportion of the variance in the test data and maintained relatively low forecast errors.

A later monthly inventory-intelligence model recorded approximately:

Metric	Recorded Value
Mean Absolute Error	14.213
Root Mean Squared Error	26.306
Coefficient of Determination	0.9353

The monthly dataset reportedly contained:

- Approximately 297,011 records
- 1,000 unique products
- Eight stores

These results must be rechecked using the final model files and final evaluation dataset before being included in the published paper. Only the latest reproducible results should appear in the final submission.

---

15. Important Features Identified by the Models

Important features observed during model development included:

Daily Forecasting

- Previous-day sales
- Seven-day rolling mean
- Festival status
- Event type
- Previous-week sales
- Product subcategory
- Store type
- City
- Weather condition
- Seasonal events

Monthly Inventory Intelligence

- Monthly units sold
- Previous two-month average
- Customer count
- Previous three-month average
- Previous six-month average
- Maximum stock
- Reorder level
- Campaign discount
- School days
- Holiday days
- Humidity
- Month
- Previous-month sales

Feature importance values should only be included in the final paper if they are extracted from the final trained model.

---

16. Functional Testing Results

The following workflows were tested during implementation:

- Forecast request submission
- Product and store retrieval
- Inventory analysis
- Recommendation generation
- Best source-store selection
- Alternative-store ranking
- Approval
- Rejection
- Cancellation
- Transfer execution
- Source-stock reduction
- Destination-stock increase
- Transaction creation
- Movement-history update
- Firebase connection
- Backend health status

The system successfully completed the main stock-transfer workflow and automatically updated the related inventory and transaction records after execution.

A formal testing table should be prepared with the following fields:

- Test case ID
- Tested function
- Input
- Expected result
- Actual result
- Pass or fail status

---

17. Research Contribution

The main contribution of this component is the combination of multiple inventory decision capabilities within one integrated system.

Unlike a basic demand forecasting application, the proposed component:

- Predicts future demand
- Interprets inventory conditions
- Detects shortage and overstock
- Identifies suitable source stores
- Compares alternative stores
- Estimates transfer distance and cost
- Evaluates risk
- Generates explainable recommendations
- Supports managerial approval
- Executes inventory transfers
- Updates stock records automatically

Therefore, the component connects machine-learning prediction with a practical operational workflow.

---

18. Current Limitations

The following limitations should be acknowledged:

- The model was primarily trained using simulated or prepared retail data.
- Full validation using live retail sales data is still limited.
- Logistics cost calculations may use simplified assumptions.
- Real-time traffic and vehicle availability are not considered.
- Forecast accuracy may change for new products with limited sales history.
- The system currently focuses on selected stores, products, brands, and product attributes.
- Generated AI explanations depend on the information provided to the language model.
- Final integration with every group component should be verified.
- The system has not yet been deployed across a complete live retail environment.
- Human approval is still required before stock transfers are executed.

---

19. Recommended Future Work

Future improvements may include:

- Real-time point-of-sale integration
- Live weather API integration
- Real-time traffic and logistics data
- Supplier lead-time prediction
- Purchase-order recommendation
- Automatic safety-stock optimisation
- New-product forecasting
- Online model retraining
- Continuous learning from actual transfers
- Mobile notifications
- Integration with warehouse systems
- Larger real-world datasets
- Advanced conversational inventory assistant
- Deeper integration with customer-behaviour and marketing signals